

VERILOG · HOMEWORK

8-bit Adder 결과의 4-Digit FND 십진수(BCD) 표시 RTL 설계 · 합성 · 검증

Basys3 (Artix-7) | 조합 회로 → FSM 스캔 → 클럭 분주 3단계 개선

131,072 전수(Exhaustive, i_cin 0/1) 시뮬레이션 + 보드 실측



8bit 합(0~510)을 4자리 FND에 동시에 표시 — 3차 개선으로 완료

목표

두 8bit 입력의 합 $i_a + i_b$ (0~510)를 BCD 4자리로 변환하여 Basys3 FND에 십진수로 표시하는 RTL을 설계·합성한다.

진행 경과

v1	조합 회로 + 버튼(i_{sel})으로 자리 선택 → 1자리씩만 표시
v2	FSM으로 Anode 자동 스캔 (100 MHz) → 고스팅(잔상) 발생
v3	클럭 분주(1 kHz) 후 스캔 → 4자리 동시 정상 표시

검증 방법 평가 요약 — 전수 입력·자동 판정은 우수. 다만 Anode 순서·자리 갱신을 확인하지 않아 **AN3 미구동·BCD4 누락 결함**을 **PASS**로 통과시킴을 결함 주입으로 확인 → 제안 TB로 보완 (p.25~33)

131,072

전수 입력 조합 (i_{cin} 0/1)

[PASS] $err_cnt = 0$

3 단계

$v1 \rightarrow v2 \rightarrow v3$

문제 발견 → 원인 분리 → 개선

$10_{ns} \rightarrow 1_{ms}$

자리당 점등 시간

(Basys3 권장 ≥ 0.25 ms 충족)

250 Hz

최종 화면 갱신 주기 (4 ms)

깜빡임·잔상 없음

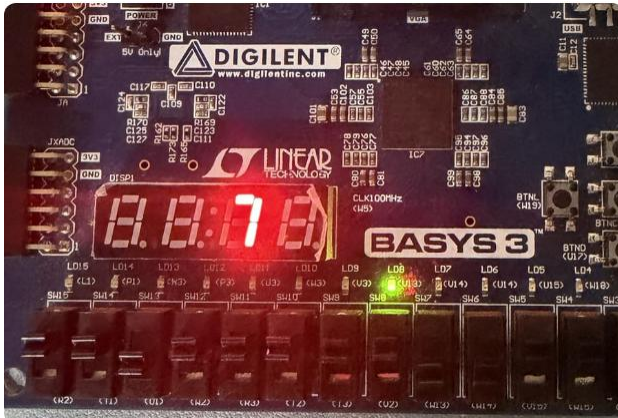
최대값 $255 + 255 = 510$ 보드 실측 표시 완료
Carry(o_cout) LED 동시 점등

원인 분석과 해결

v1 조합 회로

top_adder_fnd_8bit

- adder → digit_splitter → **mux_4x1** → fnd_decoder
- **decoder_2x4**가 버튼 i_sel로 Anode 선택
- TB 65,536 전수 **PASS**

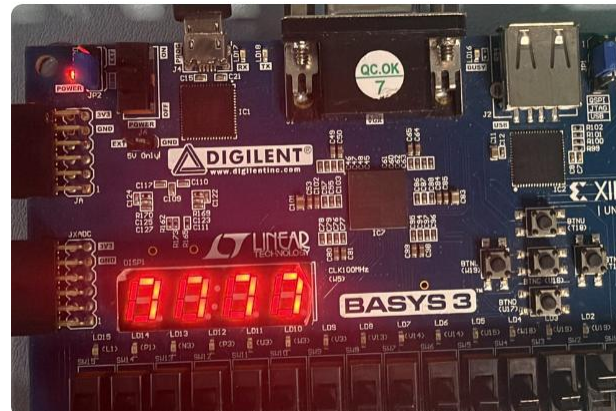


문제 — 버튼을 눌러야 한 자리씩만 보임.
합을 한눈에 읽을 수 없음

v2 FSM 자동 스캔

top_adder_fnd_for_clk (분주 전)

- mux + decoder → **Fnd_Controller**(FSM) 대체
- 100 MHz 클럭으로 AN0→AN3 순차 점등
- TB 131,072 전수 **PASS** (i_cin 0/1)

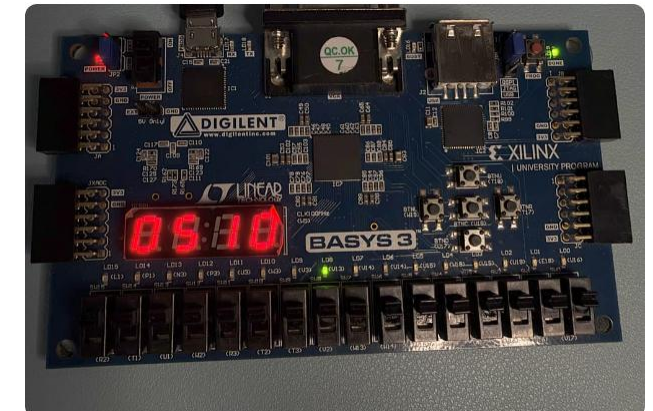


문제 — FSM 버그 수정 후에도 숫자가 옆
자리에 겹쳐 보임 (고스팅)

v3 클럭 분주 스캔

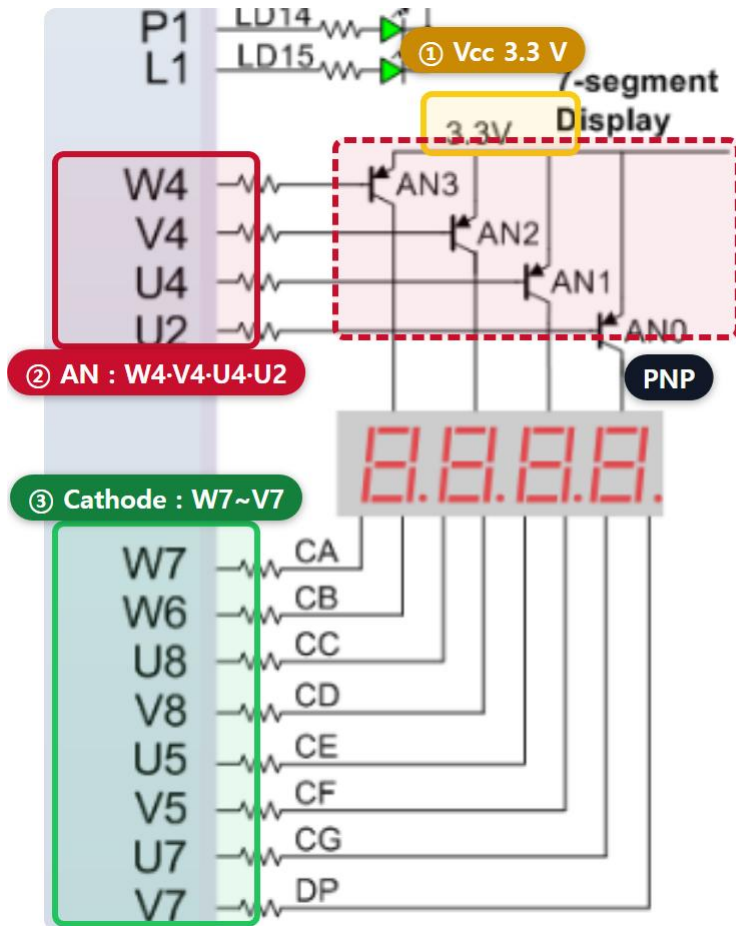
top_adder_fnd_for_clk + clock_div

- **clock_div** 추가: 100 MHz → 1 kHz
- 자리당 1 ms, 화면 갱신 4 ms (250 Hz)
- 보드 실측 정상 표시



결과 — 4자리 동시 표시, 잔상 없음. 최대
값 510 확인

Basys3 FND는 Common Anode — Anode·Cathode 모두 Active-Low



1 Emitter에 Vcc(3.3 V) 상시 인가

4자리의 Anode는 각각 PNP 트랜지스터를 통해 3.3 V에 연결

2 Base(AN) = 0 V → 해당 자리 ON

PNP는 Base가 Low여야 한다 → 따라서 `o_fnd_com`은 **Active-Low Signal**

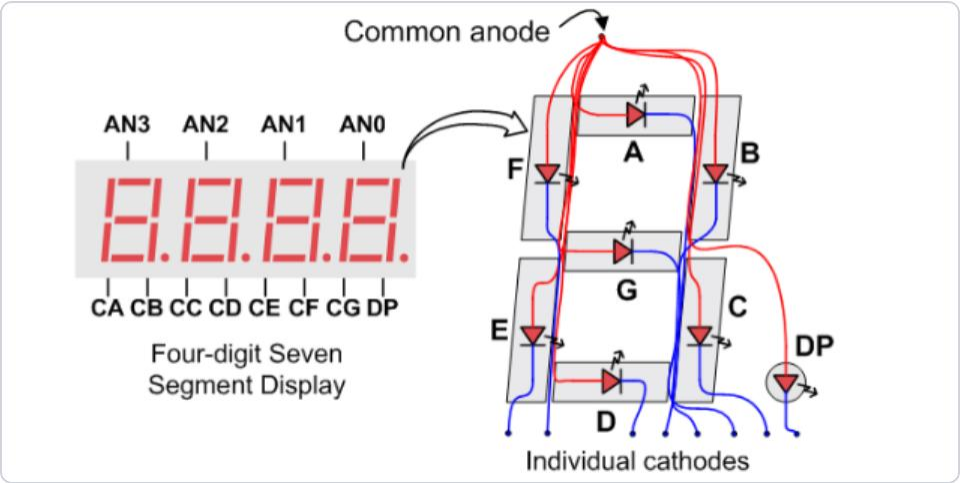
1110 = AN0(일의 자리)만 ON

3 Cathode = 0 → 해당 세그먼트 ON

8개 Cathode(CA~CG, DP)는 **4자리가 공유** → 한 순간에는 한 자리만 원하는 숫자 표시 가능

설계 시사점 — 4자리를 동시에 보이려면 Anode를 빠르게 돌아가며 켜는 **시분할 스캔**이 필수

Segment Assert condition(Cathode =0)

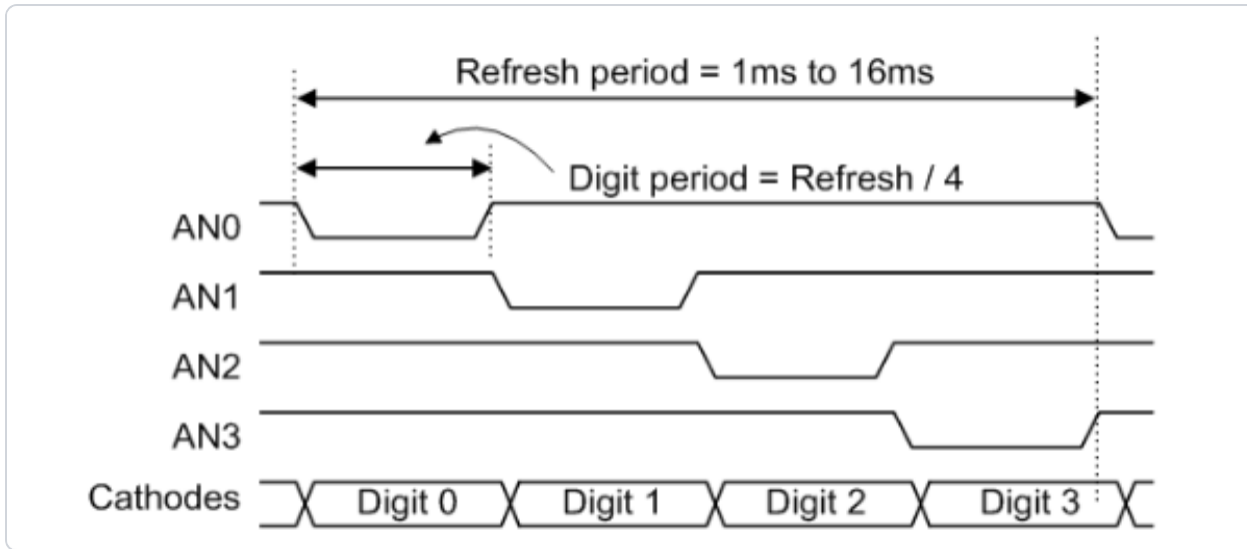


예) 숫자 '1' — B, C만 점등
DP G F E D C B A
1 1 1 1 1 0 0 1 = 8'hF9

숫자	표시	DP	G	F	E	D	C	B	A	HEX
0		1	1	0	0	0	0	0	0	C0
1		1	1	1	1	1	0	0	1	F9
2		1	0	1	0	0	1	0	0	A4
3		1	0	1	1	0	0	0	0	B0
4		1	0	0	1	1	0	0	1	99
5		1	0	0	1	0	0	1	0	92
6		1	0	0	0	0	0	1	0	82
7		1	1	1	1	1	0	0	0	F8
8		1	0	0	0	0	0	0	0	80
9		1	0	0	1	0	0	0	0	90

0 = 점등 (Active-Low) · o_fnd_data[7:0] = {DP, G, F, E, D, C, B, A} · fnd_decoder.v 기준

Time-Multiplexed Scan Timing — ≥ 0.25 ms per Digit



출처: Digilent Basys3 Reference Manual — 7-Segment Display timing

1 ~ 16 ms

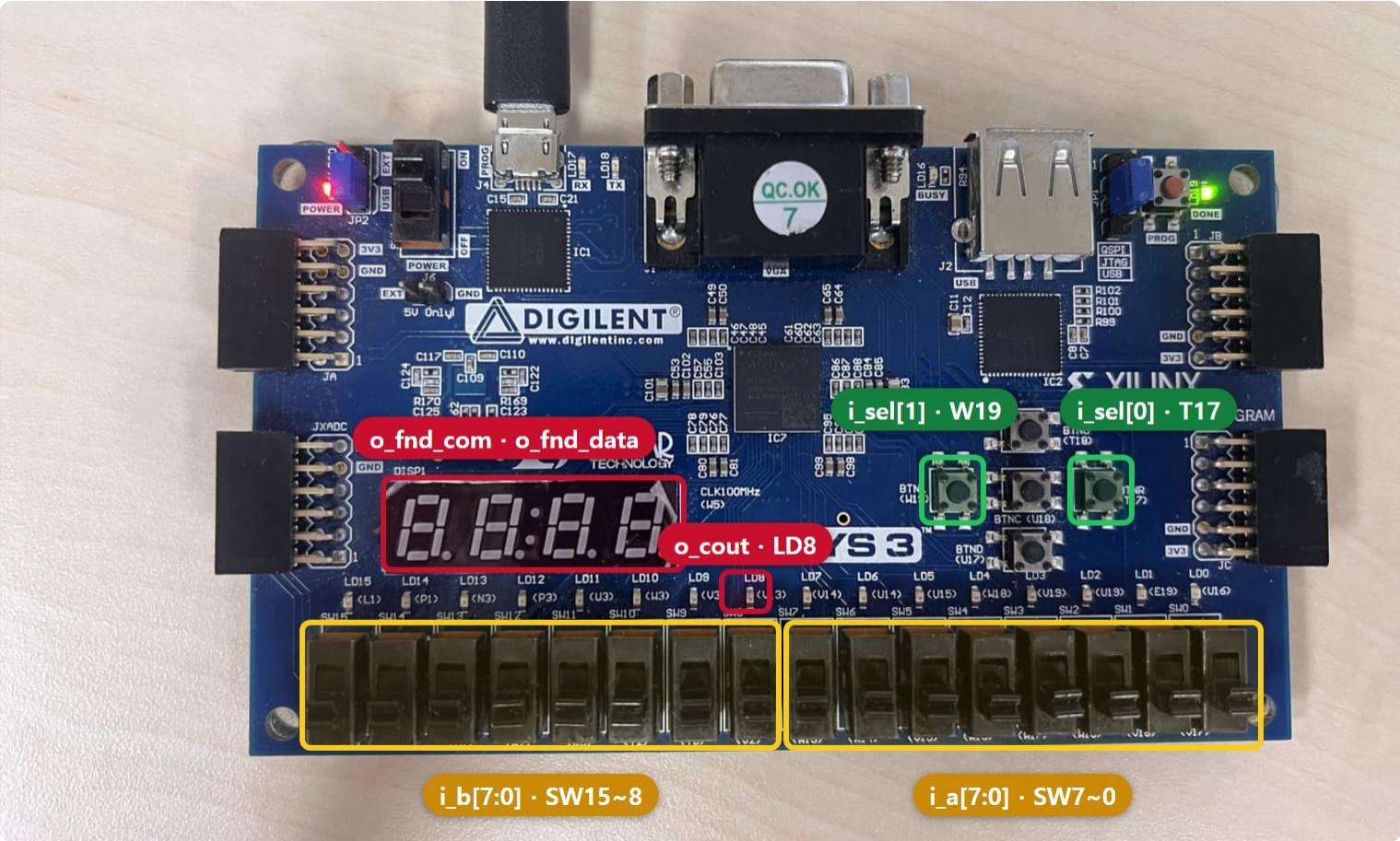
Refresh period — 4자리를 한 바퀴 도는 시간
16 ms 초과 시 깜빡임(Flicker)

≥ 0.25 ms

Digit period = Refresh / 4
이보다 짧으면 트랜지스터·LED가 따라오지 못해 잔상(고스팅)

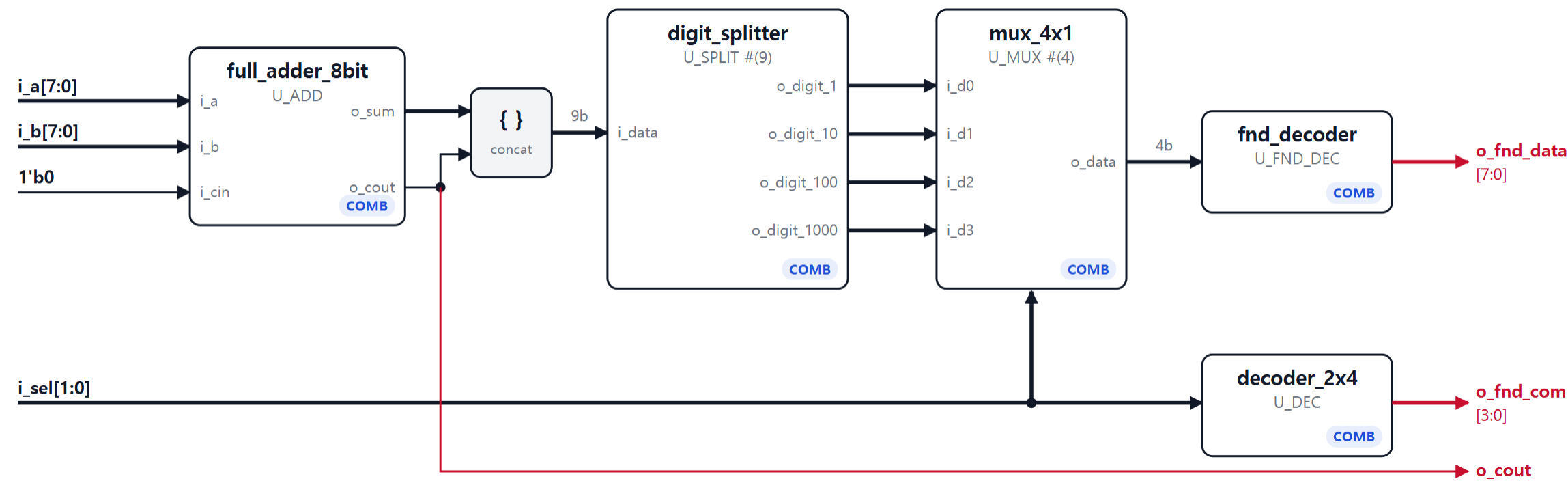
Anode 지연 — AN 전환 시 PNP 트랜지스터 On/Off 지연이 존재 →
자리당 점등 시간이 이를 충분히 덮어야 함

V1 I/O information — top_adder_fnd_8bit

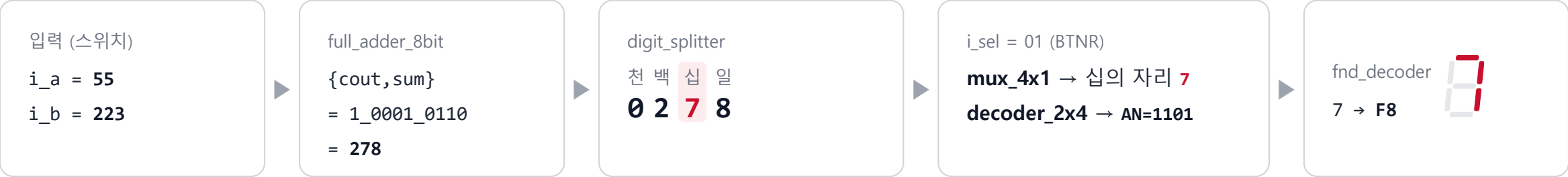


포트	보드	핀
i_a[7:0]	SW7~SW0	W13...V17
i_b[7:0]	SW15~SW8	R2...V2
i_sel[1]	BTNL	W19
i_sel[0]	BTNR	T17
o_fnd_com[3:0]	AN3~AN0	W4 V4 U4 U2
o_fnd_data[7:0]	DP, CG~CA	V7 U7...W7
o_cout	LD8	V13

V1 Block diagram

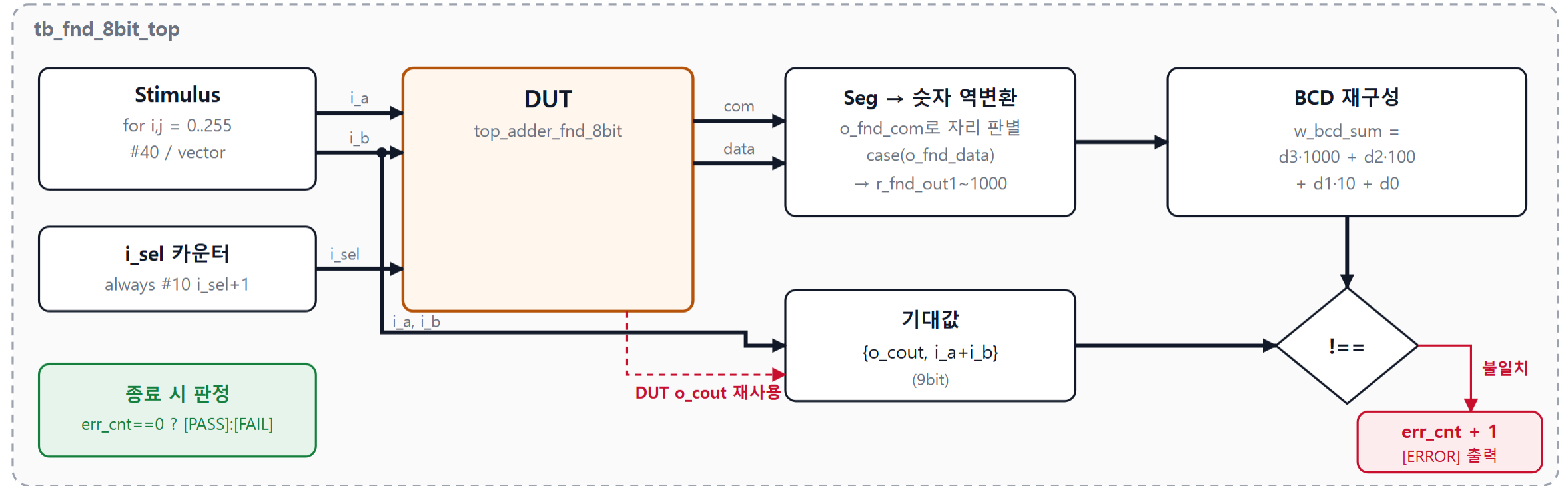


Module Roles — Data Flow of 55 + 223 = 278



모듈	역할	구현 핵심
full_adder_8bit	8bit 덧셈, 최대 255+255 = 510 (9bit 필요)	4bit RCA × 2, {o_cout, o_sum}
digit_splitter	9bit 이진수 → 천·백·십·일 자리 BCD 4개로 분리	/10, %10 연산 (WIDTH=9)
mux_4x1	4개 자리 중 지금 표시할 숫자 하나 를 선택	case(i_sel) → i_d0~i_d3
decoder_2x4	MUX와 같은 i_sel로 그 숫자가 켜질 자리(Anode) 선택	00→1110 ... 11→0111 (one-cold)
fnd_decoder	BCD 숫자(0~9) → 7-segment 폰트(Active-Low)	0→C0, 1→F9 ... 9→90

전체 65,536 케이스 검증 — 출력 핀을 역변환하여 합과 비교하는 Self-checking TB

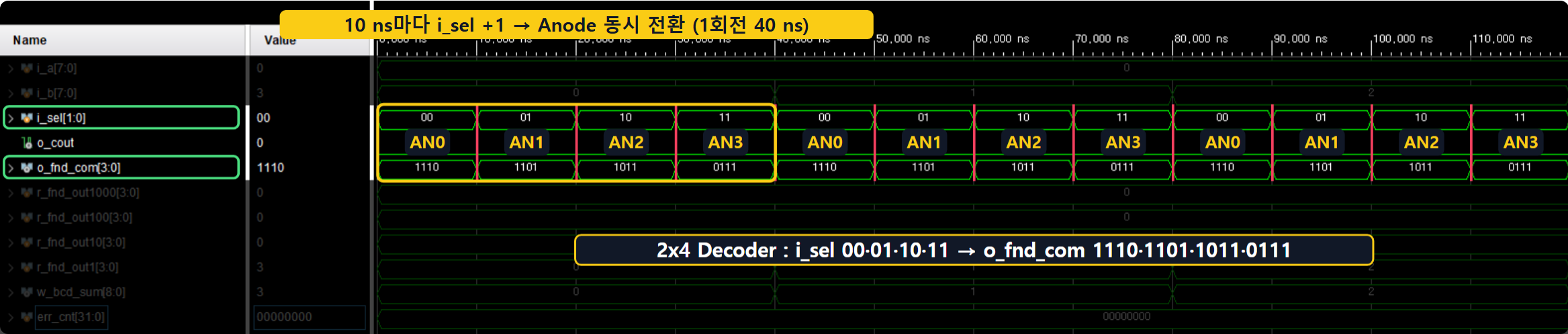


① 전수 자극 — i, j = 0~255 이중 루프, 벡터당 40 ns (= i_sel 4회전)

② 블랙박스 관측 — 내부 신호가 아닌 **FND** 핀 출력을 숫자로 역변환

③ 자동 판정 — 불일치 시 err_cnt + 1, 종료 시 **[PASS]/[FAIL]**

i_sel이 바뀌는 순간 해당 Anode 하나만 켜짐 — 2x4 Decoder 정상

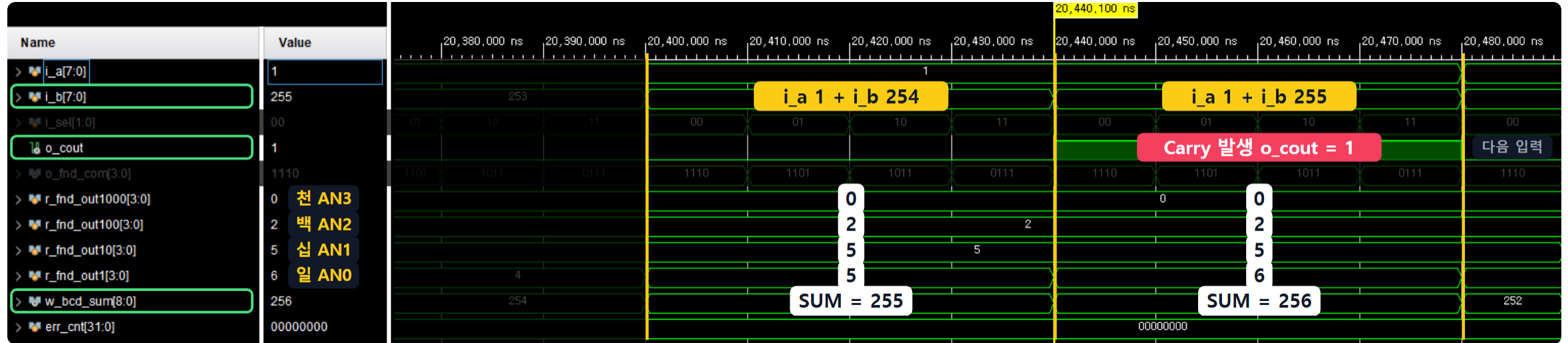


i_sel	o_fnd_com	켜지는 자리	표시 숫자 (mux)
00	1110	AN0 · 일	o_digit_1
01	1101	AN1 · 십	o_digit_10
10	1011	AN2 · 백	o_digit_100
11	0111	AN3 · 천	o_digit_1000

확인 — 10 ns ~ 110 ns 전 구간에서 i_sel 변화와 o_fnd_com 전환이 동시에 일어남 (조합 회로)

항상 한 자리만 0 (one-cold) → 두 자리가 동시에 켜지는 구간 없음

a + b = BCD 출력 — 각 자리 값과 합이 일치, Carry 구간 포함



```

2620800 ns : i_a = 255 / i_b = 240 o_cout = 1 BCD_NUM = 495
2620840 ns : i_a = 255 / i_b = 241 o_cout = 1 BCD_NUM = 496
2620880 ns : i_a = 255 / i_b = 242 o_cout = 1 BCD_NUM = 497
2620920 ns : i_a = 255 / i_b = 243 o_cout = 1 BCD_NUM = 498
2620960 ns : i_a = 255 / i_b = 244 o_cout = 1 BCD_NUM = 499
2621000 ns : i_a = 255 / i_b = 245 o_cout = 1 BCD_NUM = 490
2621010 ns : i_a = 255 / i_b = 245 o_cout = 1 BCD_NUM = 400
2621020 ns : i_a = 255 / i_b = 245 o_cout = 1 BCD_NUM = 500
2621040 ns : i_a = 255 / i_b = 246 o_cout = 1 BCD_NUM = 501
2621080 ns : i_a = 255 / i_b = 247 o_cout = 1 BCD_NUM = 502
2621120 ns : i_a = 255 / i_b = 248 o_cout = 1 BCD_NUM = 503
2621160 ns : i_a = 255 / i_b = 249 o_cout = 1 BCD_NUM = 504
2621200 ns : i_a = 255 / i_b = 250 o_cout = 1 BCD_NUM = 505
2621240 ns : i_a = 255 / i_b = 251 o_cout = 1 BCD_NUM = 506
2621280 ns : i_a = 255 / i_b = 252 o_cout = 1 BCD_NUM = 507
2621320 ns : i_a = 255 / i_b = 253 o_cout = 1 BCD_NUM = 508
2621360 ns : i_a = 255 / i_b = 254 o_cout = 1 BCD_NUM = 509
2621400 ns : i_a = 255 / i_b = 255 o_cout = 1 BCD_NUM = 500
2621410 ns : i_a = 255 / i_b = 255 o_cout = 1 BCD_NUM = 510
[PASS] : ALL BCD CORRECT!
  
```

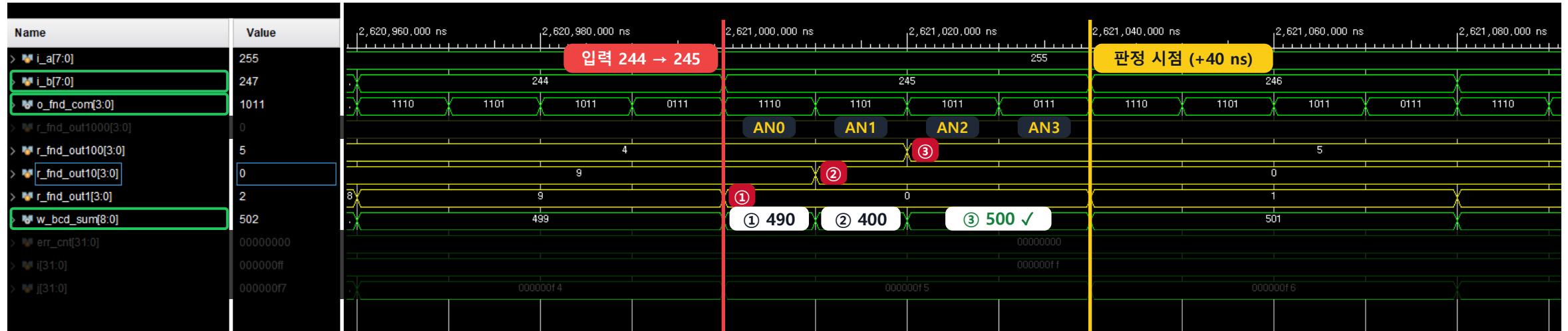
판독 — 1 + 254 → 천·백·십·일 = 0 2 5 5 = 255

1 + 255 → 천·백·십·일 0 2 5 6 = 256 → o_cout = 1

[PASS] ALL BCD CORRECT — 255 + 255 = 510까지 65,536 벡터 일치 (Tcl Console)

로그의 490 → 400 → 500은 입력이 바뀐 직후 자리가 하나씩 갱신되는 과도 구간 값. 판정은 40 ns 뒤에 하므로 오류 아님 → 해석은 다음 장

로그의 490 → 400 → 500 — 자리가 하나씩 갱신되는 과도값, 판정 시점엔 500



① 2,621,000 ns · AN0

일의 자리만 새 값(9→0)
백·십은 이전 입력값 유지

490 4 · 9 · 0

② 2,621,010 ns · AN1

십의 자리 갱신(9→0)
백은 아직 이전값 4

400 4 · 0 · 0

③ 2,621,020 ns · AN2

백의 자리 갱신(4→5)
이제 4자리 모두 새 값

500 5 · 0 · 0

④ 2,621,040 ns · 판정

입력 후 #40 = Anode 1바퀴
{o_cout, i_a+i_b} 와 비교

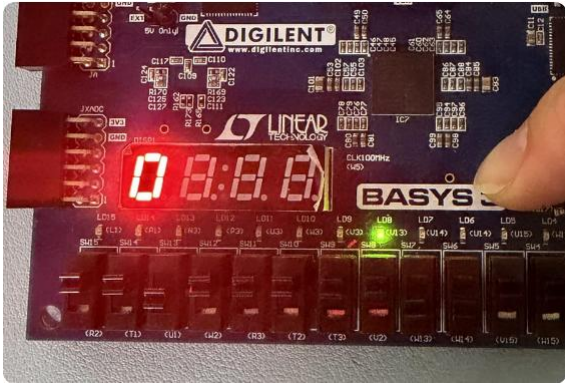
500 = 255 + 245 ✓ PASS

왜 이렇게 찍히나

— TB는 `o_fnd_com`이 가리키는 자리 하나만 역변환해 `r_fnd_out*`에 저장한다. 입력이 바뀌면 AN0 → AN3 순서로 10 ns마다 한 자리씩 새 값이 되고, \$monitor는 `w_bcd_sum`이 바뀔 때마다 출력하므로 중간값 490 · 400이 로그에 남는다.

판정 — 4자리가 모두 갱신된 #40 시점에만 비교 → 과도값은 판정 대상이 아님 (65,536 벡터, `err_cnt` = 0)

보드 실측 — $55 + 223 = 278$, 버튼 조합으로 한 자리씩 확인



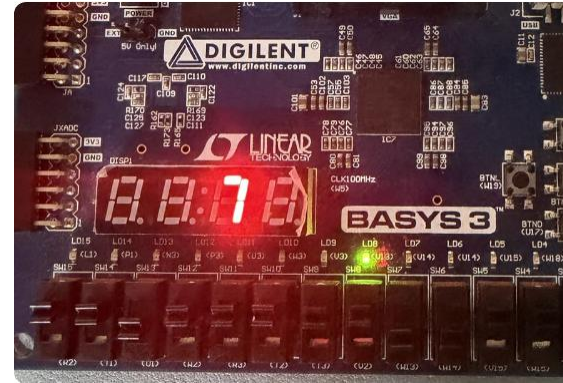
$i_sel = 11$
BTNL + BTNR → AN3 · 천



$i_sel = 10$
BTNL → AN2 · 백



$i_sel = 01$
BTNR → AN1 · 십



$i_sel = 00$
누르지 않음 → AN0 · 일

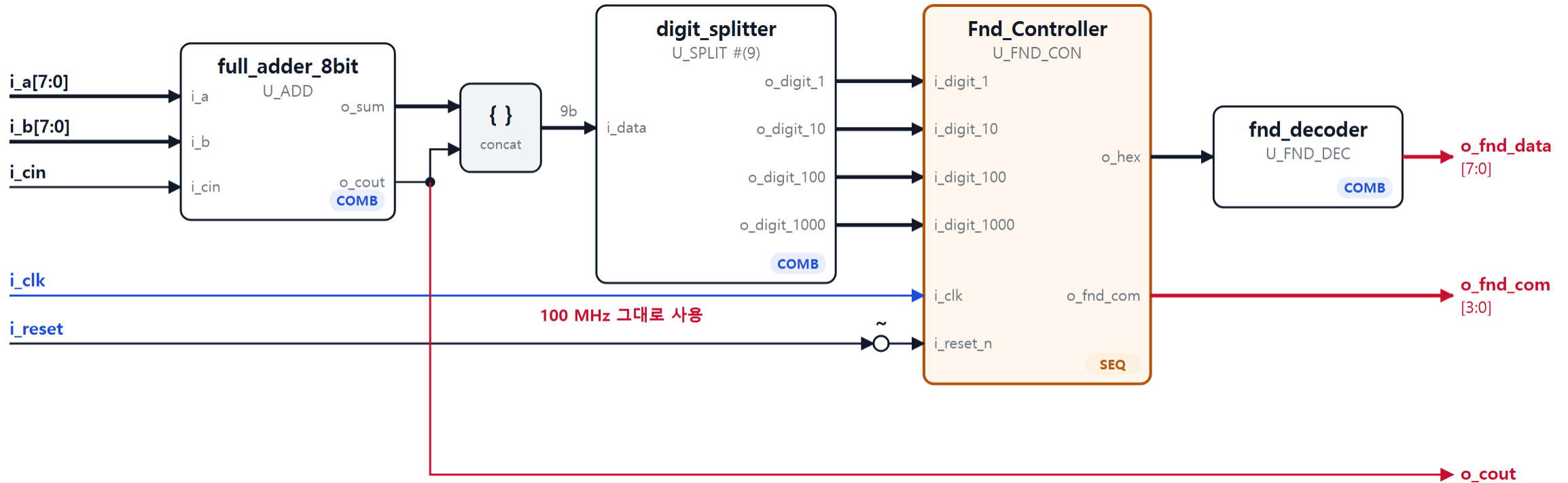


0278

판독 — 4장을 합치면 278 ✓
 $278 \geq 256$ 이므로 LD8(o_cout) 점등 ✓

트러블슈팅 #1 — W19·T17 버튼을 매번 눌러 Anode를 바꿔야 하므로 합을 한눈에 읽을 수 없음

개선 #2 — MUX·Decoder를 FSM으로 대체, Anode 자동 순환

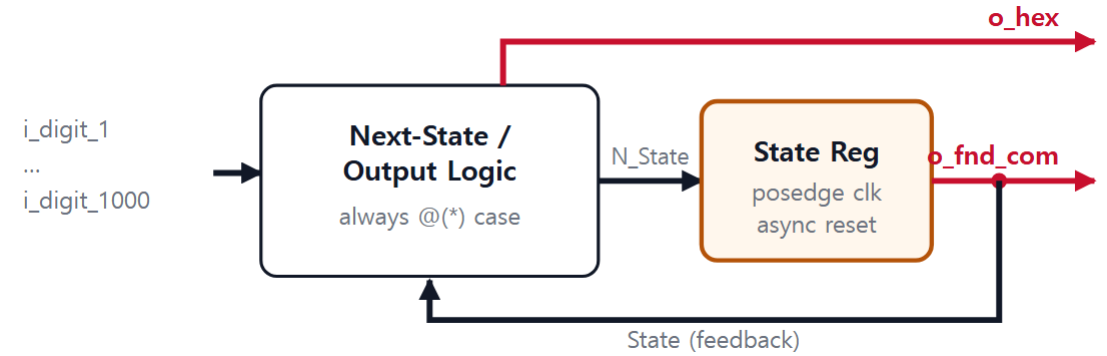
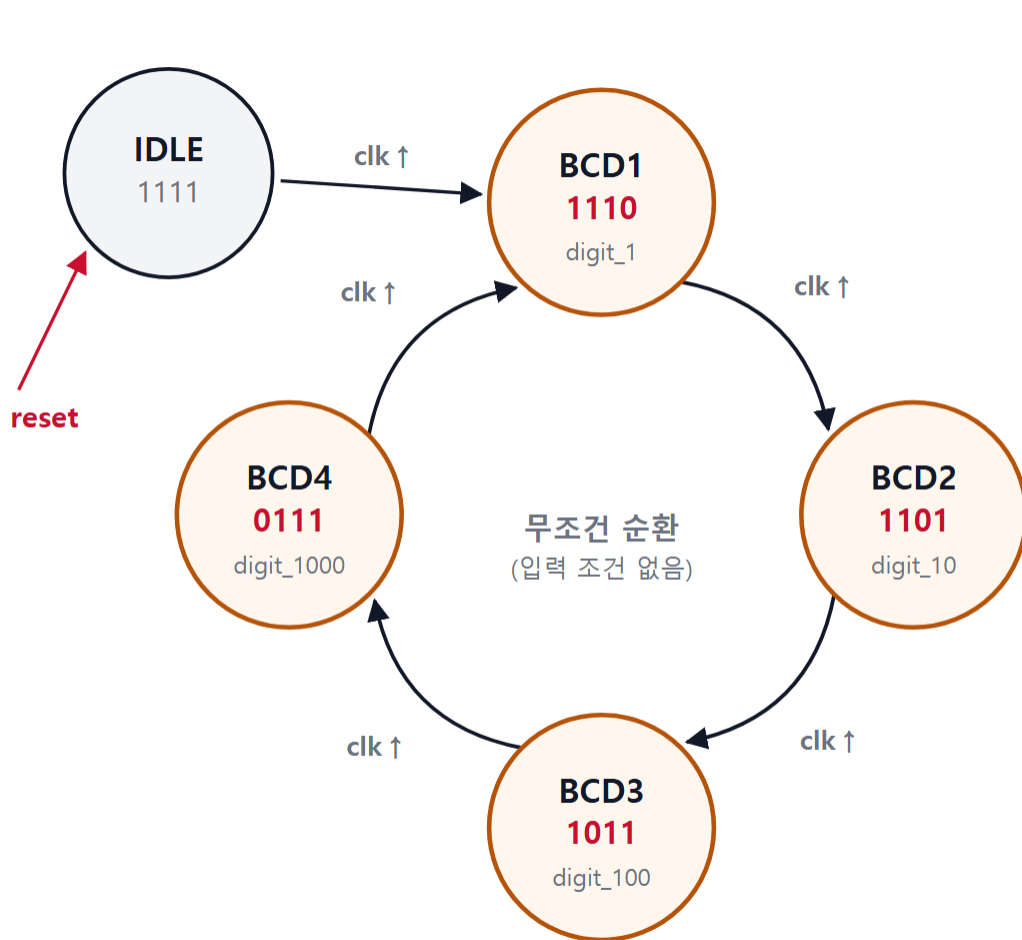


COMB 조합 회로

SEQ 순차 회로 (신규)

버튼 입력 i_{sel} 제거 → 사람 대신 FSM이 자리 선택

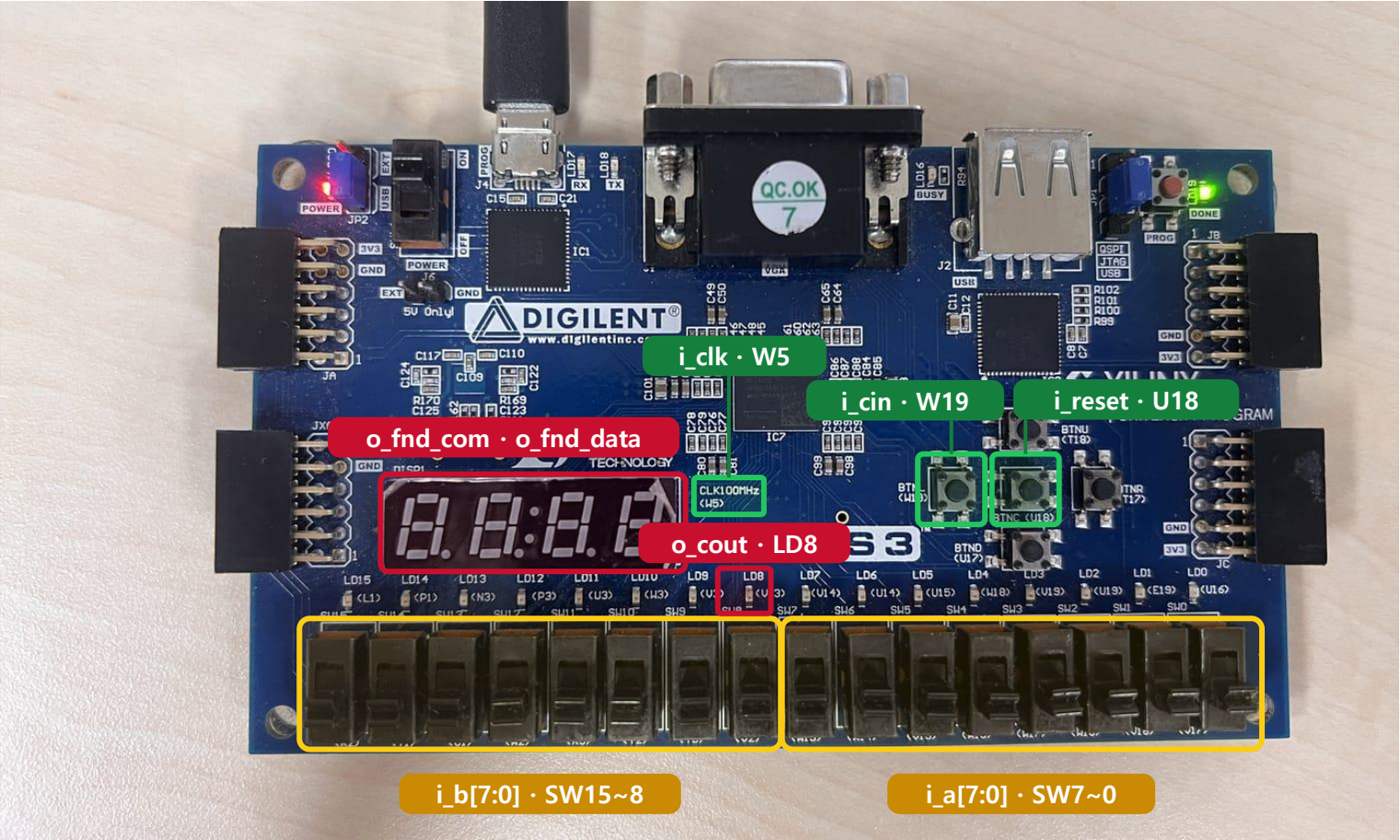
Fnd_Controller 내부 — State 값 자체가 Anode 출력이 되는 4-State 순환 FSM



State	값 = o_fnd_com	o_hex	다음
IDLE	1111 (모두 OFF)	4'hF	BCD1
BCD1	1110 (AN0)	i_digit_1	BCD2
BCD2	1101 (AN1)	i_digit_10	BCD3
BCD3	1011 (AN2)	i_digit_100	BCD4
BCD4	0111 (AN3)	i_digit_1000	BCD1

설계 포인트 — State 인코딩 = Anode 패턴(one-cold)이라 출력 디코딩 로직이 없음 → Anode에 조합 글리치 없음. 출력이 State에만 의존하므로 **Moore형** 구조

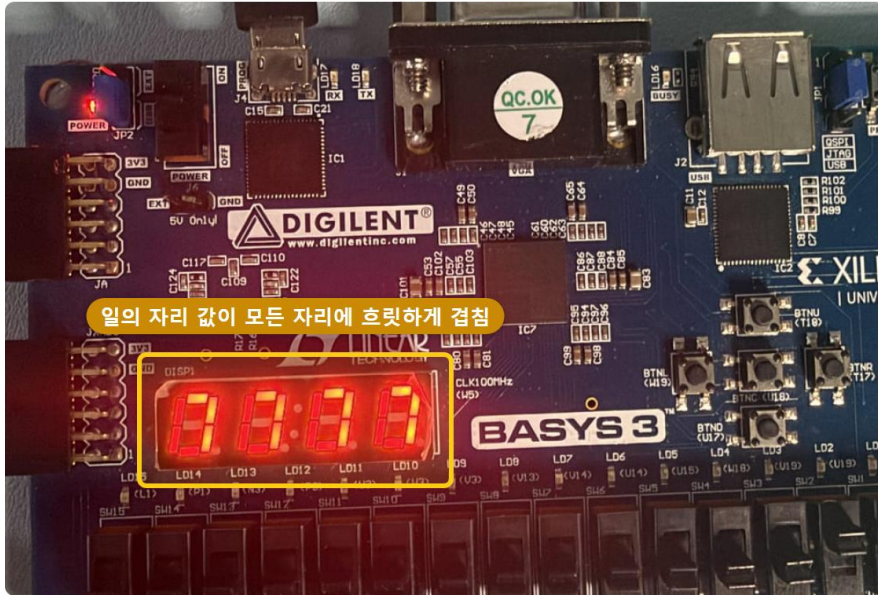
v2 입출력 포트 — 클럭·리셋·Carry-in 추가, 자리 선택 버튼 제거



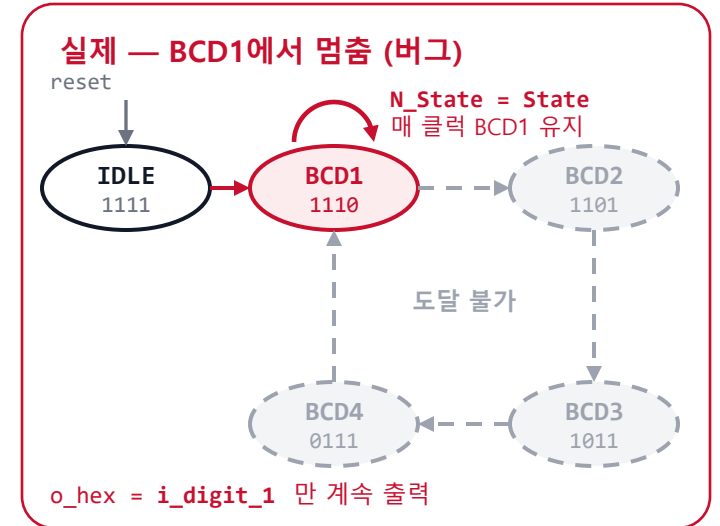
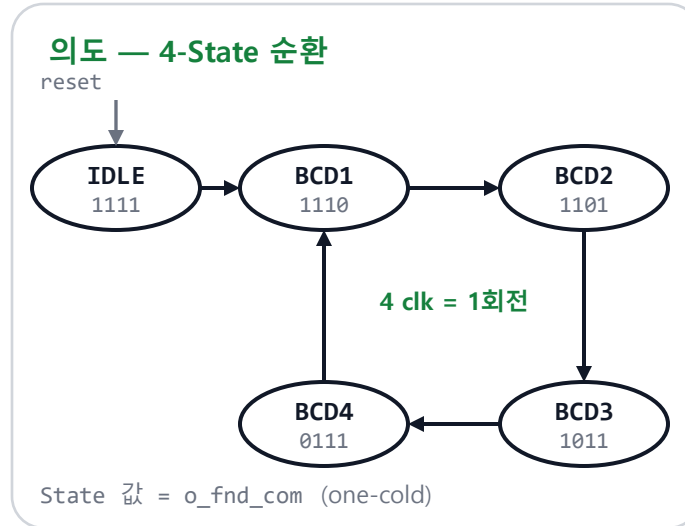
포트	보드	핀
i_clk	100 MHz OSC	W5
i_reset	BTNC	U18
i_cin	BTNL	W19
i_a / i_b	SW7~0 / SW15~8	v1과 동일
o_fnd_com / data	AN / Cathode	v1과 동일
o_cout	LD8	V13

Reset을 Active-High로 사용한 이유
BTNC(U18)는 누를 때 1 → Top에서 반전 (~i_reset_n)하여 하위 모듈의 Active-Low 리셋에 연결

보드에 일의 자리 값만 표시 — FSM Next-State 코딩 오류로 BCD1 고정



V2 첫 보드 출력 — TB 확인 없이 합성·다운로드한 결과

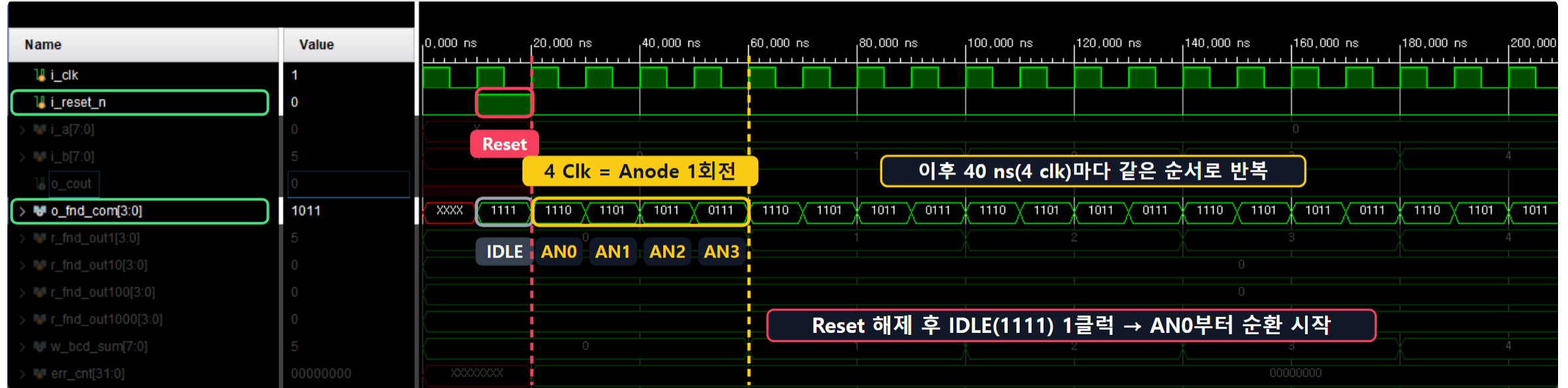


원인 — Next-State 로직에서 항상 **N_State = State**가 적용되어 case 문의 천이가 반영되지 않음 → IDLE 다음 BCD1에서 벗어나지 못함 (단순 코딩 실수)

교훈 — TB를 먼저 돌리지 않고 보드부터 출력함. 파형에서 o_fnd_com이 1110에 멈춘 것이 바로 보이므로, TB를 선행했다면 보드 출력 전에 걸러낼 수 있었던 오류

수정 후 재검증 — V1과 동일한 Self-checking TB: o_fnd_com으로 자리 판별 → o_fnd_data 역변환 → BCD_NUM 복원 → i_a+i_b+i_cin과 비교 (i_cin 0/1) → 결과 p.19 ~ 21

Reset 해제 후 IDLE → AN0~AN3가 4 클럭마다 한 바퀴씩 순환

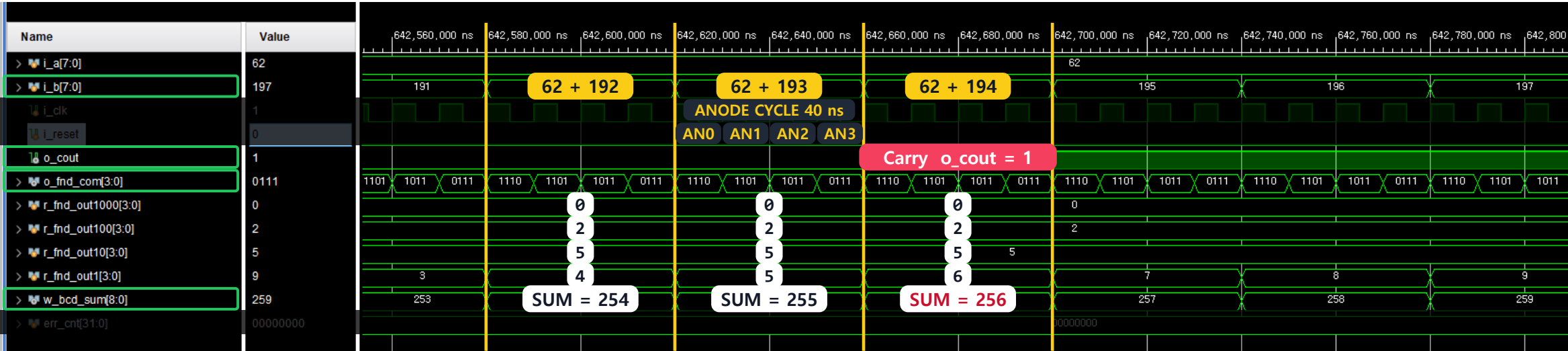


Reset — 버튼(BTNC) High 동안 State = IDLE(1111), 모든 Anode OFF

순환 — 1110 → 1101 → 1011 → 0111, 클럭마다 한 자리만 ON

Active-High Reset — BTNC(U18)는 누를 때 1 → Top에서 반전해 하위 모듈에 연결

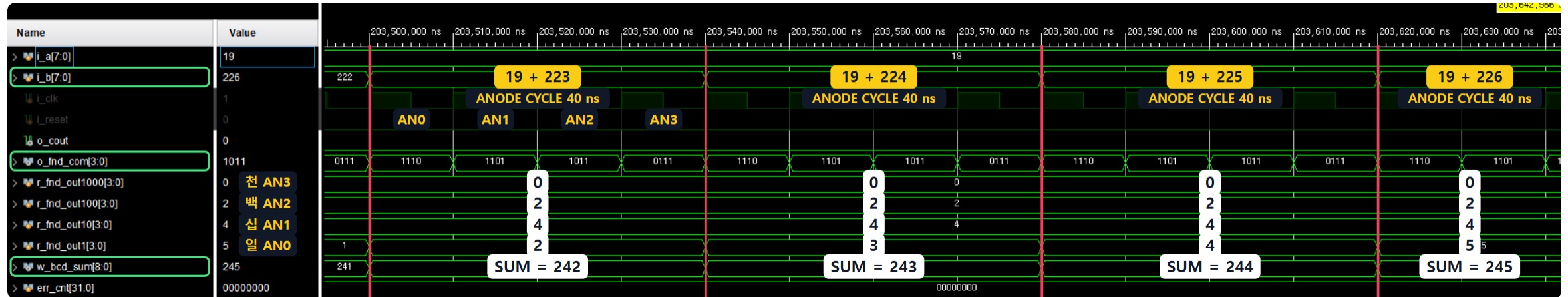
AN이 바뀔 때마다 해당 자리 숫자로 갱신 — 255 → 256 Carry까지



판독 62 + 192 → 0 2 5 4 = 254 · 62 + 193 → 0 2 5 5 = 255
62 + 194 → 0 2 5 6 = 256 → **o_cout = 1** (256부터 Carry)

TB 수정 — w_bcd_sum이 입력 구간과 같은 시점에 갱신되도록 정렬 → 이전 캡처의 "한 구간 늦게 표시" 해소, 파형의 SUM이 곧 그 입력의 결과

입력 1개당 ANODE CYCLE 1회 — 출력 확인



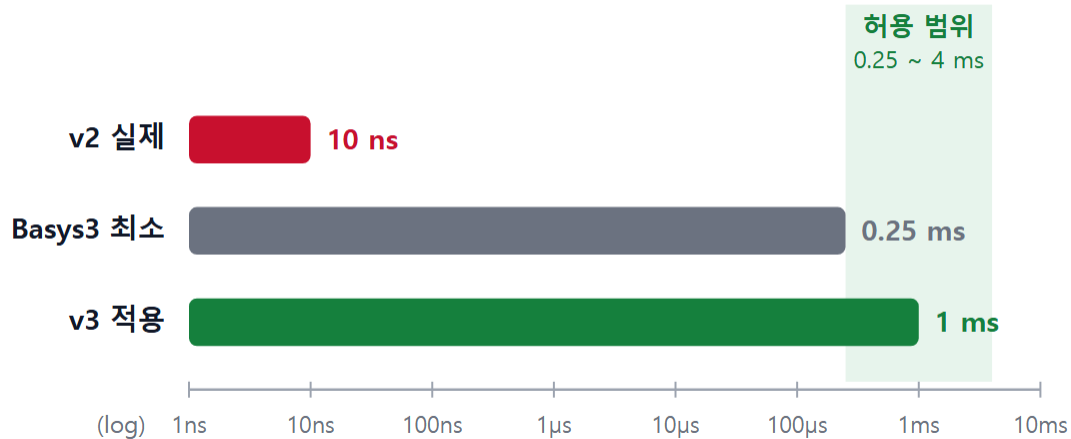
```

5242440 ns : i_a = 255 / i_b = 244 o_cout = 1 BCD_NUM = 500
5242460 ns : i_a = 255 / i_b = 245 o_cout = 1 BCD_NUM = 501
5242500 ns : i_a = 255 / i_b = 246 o_cout = 1 BCD_NUM = 502
5242540 ns : i_a = 255 / i_b = 247 o_cout = 1 BCD_NUM = 503
5242580 ns : i_a = 255 / i_b = 248 o_cout = 1 BCD_NUM = 504
5242620 ns : i_a = 255 / i_b = 249 o_cout = 1 BCD_NUM = 505
5242660 ns : i_a = 255 / i_b = 250 o_cout = 1 BCD_NUM = 506
5242700 ns : i_a = 255 / i_b = 251 o_cout = 1 BCD_NUM = 507
5242740 ns : i_a = 255 / i_b = 252 o_cout = 1 BCD_NUM = 508
5242780 ns : i_a = 255 / i_b = 253 o_cout = 1 BCD_NUM = 509
5242820 ns : i_a = 255 / i_b = 254 o_cout = 1 BCD_NUM = 500
5242830 ns : i_a = 255 / i_b = 254 o_cout = 1 BCD_NUM = 510
5242860 ns : i_a = 255 / i_b = 255 o_cout = 1 BCD_NUM = 511
[PASS] : i_cin = 1 -> ALL BCD CORRECT!
[PASS] : ALL BCD CORRECT! (i_cin = 0, 1)
  
```

4자리 값을 숫자로 환산해 비교, 틀리면 **err_cnt + 1** → 끝까지
err_cnt = 0, 131,072 케이스 (i_cin 0/1 각 65,536) **[PASS]**

결론 — FSM 수정(BCD1 고정 해소) 후 BCD 값 전수 일치 → 남은
 고스팅의 원인은 **보드 구동 타이밍**

자리당 점등 시간 10 ns — 하드웨어 요구치보다 25,000배 짧음



100 MHz로 Anode를 바꾸면 PNP 트랜지스터·LED가 꺼지기 전에 다음 자리가 켜짐
→ 특정 자리의 숫자 잔상이 옆자리에 흐릿하게 겹침 (고스팅)

해결 방안 — 100 MHz 클럭 분주 → 1 kHz (주기 1 ms)

카운터 값	출력
0 ~ 49,999	1
50,000 ~ 99,999	0
100,000 클럭 = 1 ms	→ 반복

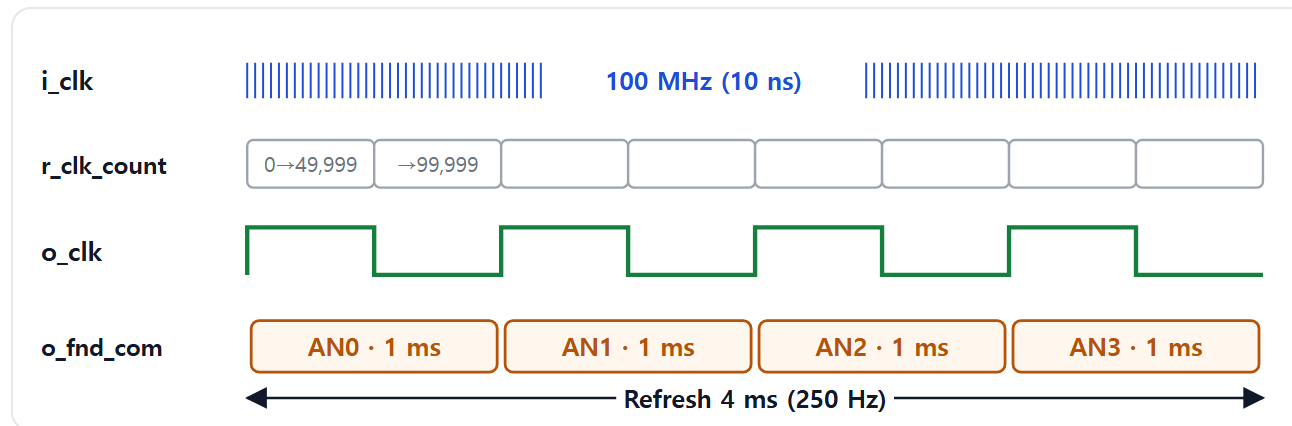
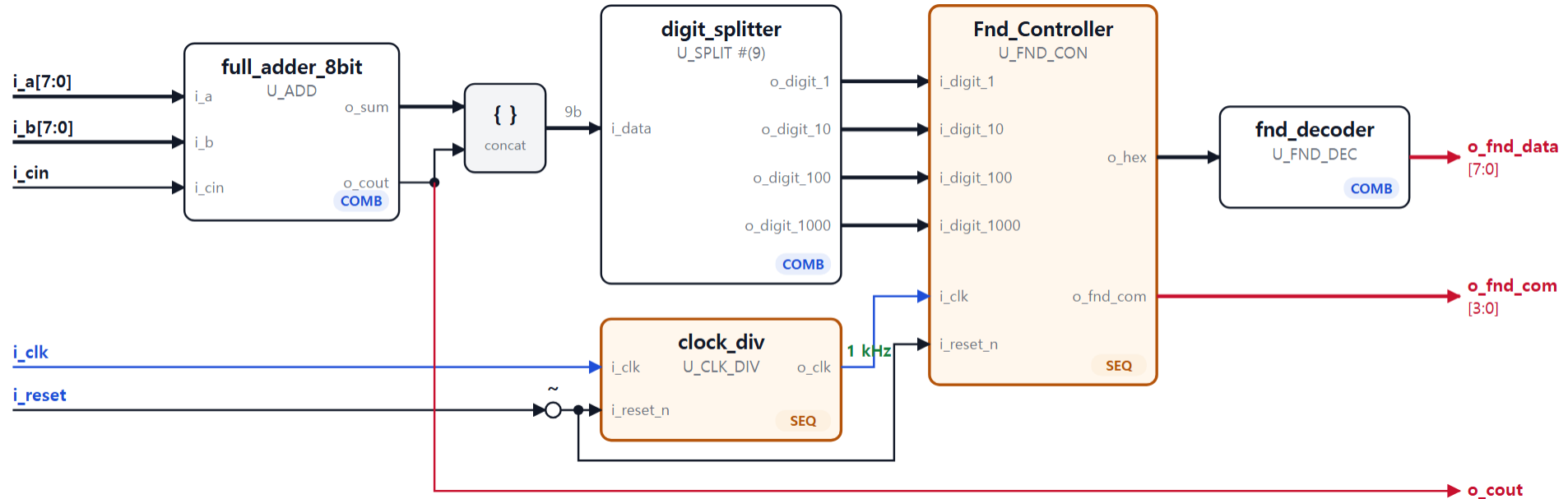
1 ms

자리당 점등 (≥ 0.25 ms ✓)

4 ms

화면 갱신 (1~16 ms ✓)

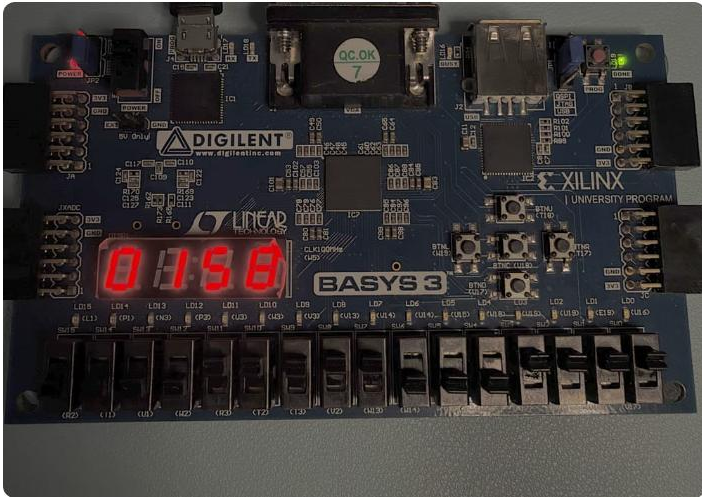
clock_div 추가 — FSM을 1 kHz로 구동하여 자리당 1 ms 점등



clock_div

- 20bit 카운터 r_clk_count로 100 MHz 클럭 계수
- 50,000 미만 → 1, 이상 → 0 : **Duty 50 %, 1 kHz**
- 출력 o_clk_100Hz가 Fnd_Controller의 클럭으로 입력

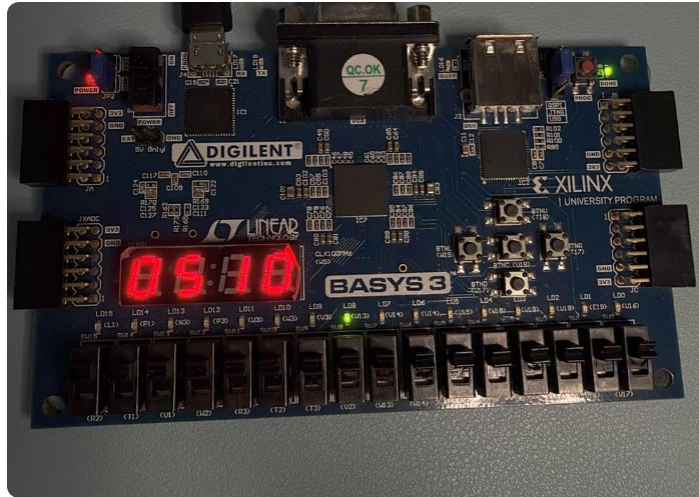
v3 보드 실측 — 4자리 동시 표시, 잔상 없음



FND 표시

0158

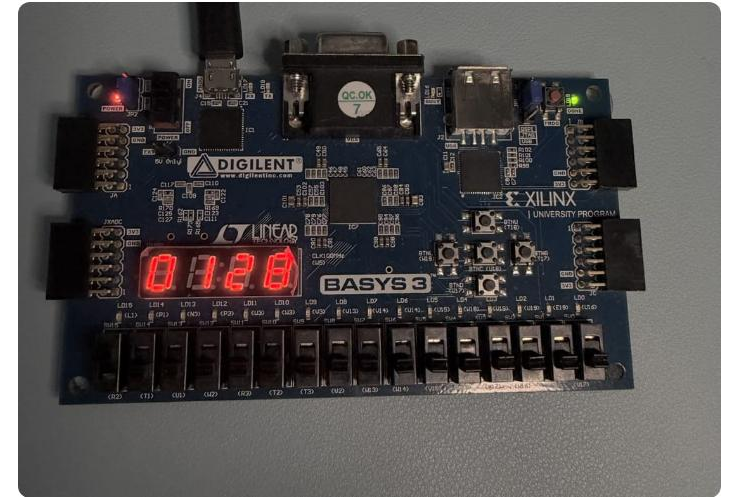
$i_a = 143$ $i_b = 15$
 $o_{cout} = 0$ · 합 < 256



FND 표시

0510

$i_a = 255$ $i_b = 255$
 $o_{cout} = 1$ (LD8 ON) · 최대값 510



FND 표시

0128

$i_a = 0$ $i_b = 128$
 $o_{cout} = 0$ · 합 < 256

확인 — 3개 입력 모두 $i_a + i_b$ 와 일치 · 버튼 조작 없이 합이 바로 읽힘 · 최대값 510에서만 Carry LED(LD8) 점등 · 1 kHz 스캔으로 깜빡임·잔상 없음

평가 — 전수·자동 판정은 우수, ‘출력 프로토콜 관측’에 허점

평가 항목	판정	근거
입력 공간 커버리지	우수	clk TB: i_cin 0/1 × 256 × 256 = 131,072 전수 자극 (v1 TB 65,536, i_cin 고정)
자동 판정 (Self-checking)	우수	err_cnt 누적 + 종료 시 [PASS]/[FAIL], 오류 시 입력값과 표시값을 함께 출력
관측 지점	우수	내부 신호가 아닌 FND 핀 출력을 역변환 → 실제 보드가 보여줄 값을 검증
기대값 독립성	양호	clk TB는 i_a+i_b+i_cin으로 독립 계산 → 덧셈 올림 고장(버그 ①) 검출. 단 LED 출력은 미비교
출력 동작 확인 (Anode)	미흡	한 자리씩·순서대로·4자리 모두 켜지는지 확인 없음 → 버그 ②④ 미검출 (p.26)
검증 대상 일치	보완 필요	보드에 올린 v3(clock_div 포함)가 아닌 _test Top만 시뮬레이션
판정 신뢰성	보완 필요	고정 #40 판정, 입력 변경·리셋 해제·판정이 clk edge와 같은 시각(실행 순서 의존), 복원 9bit
문제 해결 절차	양호	보드 이상 → TB로 RTL 확인 → 물리 타이밍으로 원인 축소. 단 TB 없이 보드 출력이 먼저여서 FSM 버그(BCD1 고정)를 사전에 못 거름

종합 (우수 5 · 양호 4 · 보완 3 · 미흡 2)

31 / 40

B — Anode 확인 보강 필요

강점 — 전수 입력 + 핀 레벨 역변환 + 자동 판정. 보드 이상을 TB로 RTL과 분리한 문제 해결 절차

핵심 약점 — 화면이 한 자리씩, 순서대로, 4자리 모두 켜지는지 확인하지 않음

기존 TB 허점

방법 회로에 버그를 하나씩 넣고 같은 입력 131,072개를 돌려, TB가 FAIL을 내는지 확인 · 보드 화면 예시는 $255 + 255 = 510$

심은 버그	보드 화면 (정상이면 0510)	기존 TB	새 TB
정상 회로	0510	PASS	PASS
① 덧셈 올림(Carry) 고장	0254 510 → 254, LED 꺼짐	✓ 잡음	✓ 잡음
② 천의 자리가 안 켜짐	8510 맨 앞 0이 안 보임	✗ 놓침	✓ 잡음
③ 첫 자리에서 멈춤	8880 일의 자리만 보임	✓ 잡음	✓ 잡음
④ 천의 자리 순서 건너뛰	8510 3자리만 돌아감	✗ 놓침	✓ 잡음
⑤ 두 자리가 동시에 켜짐	0510 일의 자리에 십의 자리가 겹침	△ 일부 놓침	✓ 잡음

왜 놓쳤나 기존 TB는 '최종 숫자 값'만 비교한다. 천의 자리는 원래 0이라 꺼져서 안 보여도 값(510)은 같다 → PASS

기존 TB 허점 6가지 → 새 TB에서는 이렇게 확인

기존 TB가 보지 않던 것		새 TB가 확인하는 것	잡는 버그
1	4자리가 다 켜졌는지 안 봄	입력마다 4자리를 전부 다시 읽은 뒤에만 채점	② ③ ④ ⑤
2	켜지는 순서를 안 봄	AN0 → AN1 → AN2 → AN3 순서를 매번 확인	② ④ ⑤
3	두 자리가 동시에 켜져도 모름	한 번에 한 자리만 켜지는지 확인	② ⑤
4	LED(Carry) 출력은 안 봄	LED 값도 정답과 따로 비교	①
5	이전 입력의 값이 남아 있어도 통과	입력이 바뀌면 기록을 지우고 처음부터 다시 읽음	② ④
6	40 ns마다 무조건 채점	4자리를 다 읽은 순간 채점 → 스캔 속도가 달라도 그대로 사용	보드 설정

그대로 둔 장점 i_cin 0/1 포함 131,072개 전부 입력 · 정답은 입력만으로 계산 (회로 출력을 정답에 쓰지 않음)

개선점

검증 (TB) — 현재 clk TB 상태 → 새 TB

반영됨

독립 기대값 + i_cin 전수

i_a+i_b+i_cin, 131,072 전수 (사용자 수정) → 버그 ① 검출. LED는 새 TB에서 따로 비교

미흡

Anode 프로토콜 확인

한 자리씩 · 순서대로 · 4자리 모두 켜지는지 확인 → 새 TB에서 버그 ② ④ ⑤ 검출

보완 필요

복원 폭 · 무효 폰트

읽은 값을 넉넉한 폭(14bit)으로 복원, 0~9 이외 모양은 즉시 오류 → 새 TB 구현

RTL (설계 관점)

보완 필요

분주 클럭을 클럭으로 사용

로직으로 만든 클럭이 FSM을 구동 → 전용 클럭망(BUFG) 미사용, 클럭 도메인 증가. 100 MHz 단일 클럭 + 1 kHz Clock Enable 권장

양호

비동기 입력 처리

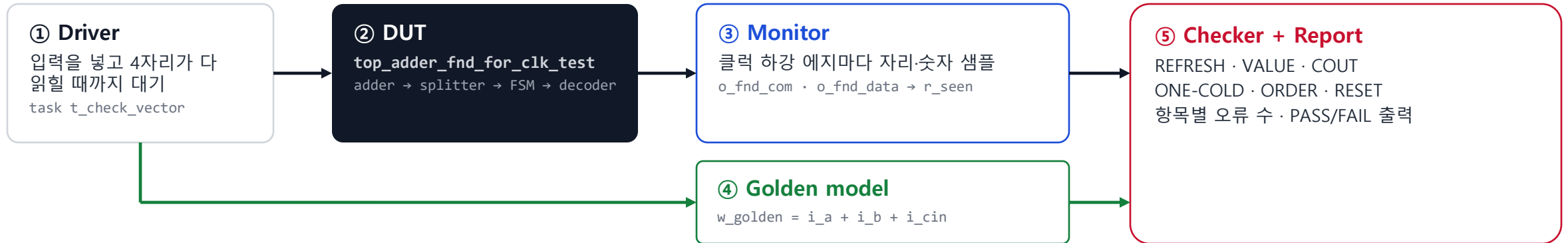
고찰의 “값 흔들림”은 2-FF 동기화 + 디바운스, 프레임 경계 래치로 해결 가능

적용 우선순위

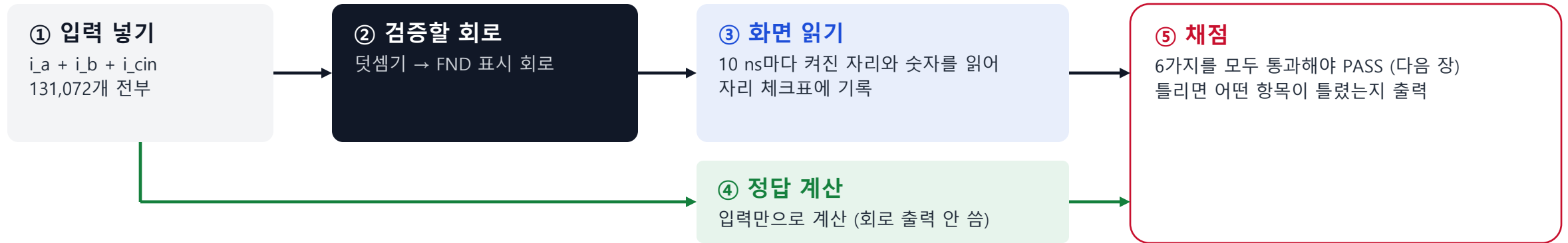
- 1 Anode 순서·갱신 확인 — 새 TB로 버그 ② ④ ⑤ 검출 (p.26~27)
- 2 V3 최종 Top 시뮬레이션 — 같은 TB의 V3 모드로 보드와 같은 구조 검증
- 3 Clock Enable 구조 전환 — 단일 클럭 도메인으로 타이밍 분석 단순화

새 TB 구조 — 모듈 구성과 쉬운 풀이

TB 모듈 구성 (코드 기준)



쉽게 말하면 — 같은 번호끼리 같은 역할



왜 1 ms가 아니라 10 ns로? 보드는 자리당 1 ms(1 kHz)로 스캔하지만, 그 속도로 131,072개를 전부 돌리면 시뮬레이션에 오래 걸림(Vivado 시뮬레이터 속도로 환산시 4.9시간). 그래서 분주기를 뺀 시뮬레이션용 Top(자리당 10 ns, 약 10초)으로 전수 검증한다.

채점에서 확인하는 6가지 — 화면이 어떻게 켜지는지까지 본다

1 한 번에 한 자리만 켜지나?

두 자리가 같이 켜지면 숫자가 겹쳐 보임

예) 1110 ✓

예) 1100 ✗

잡는 버그 ② ⑤

2 AN0 → AN1 → AN2 → AN3 순서인가?

자리를 건너뛰거나 거꾸로 돌면 오류

예) AN2 → AN3 ✓

예) AN2 → AN0 ✗

잡는 버그 ② ④ ⑤

3 제한 시간 안에 4자리가 다 켜지나?

안 켜지는 자리, 멈춘 자리를 찾아냄

예) 40 ns 안에 4칸 ✓

예) 못 채움 ✗

잡는 버그 ② ③ ④ ⑤

4 화면 숫자 = 정답인가?

4자리를 다 읽은 뒤 정답과 비교

예) 0510 = 255 + 255 ✓

잡는 버그 ①

5 LED(Carry) = 정답의 올림인가?

합이 256 이상일 때만 켜져야 함

예) 510 → LED ON ✓

예) 254 → LED OFF ✓

잡는 버그 ①

6 리셋 중엔 모두 꺼지나?

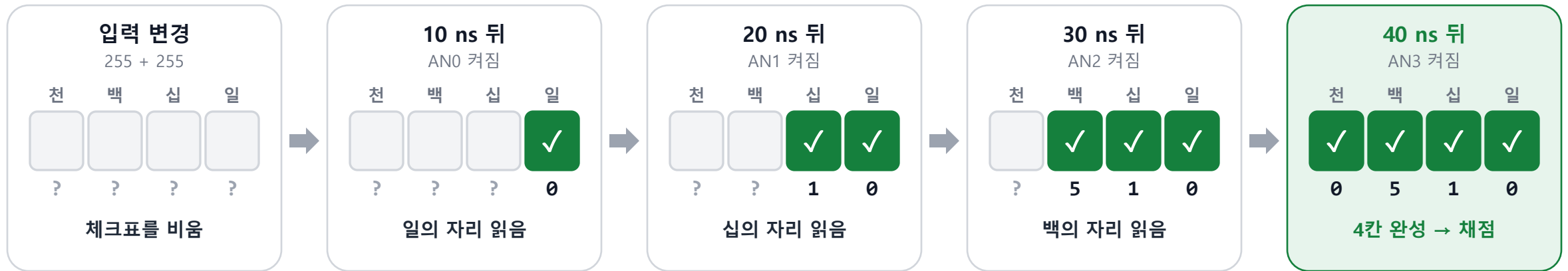
리셋 버튼을 누르는 동안 4자리 OFF

예) 1111 (모두 OFF) ✓

하나라도 틀리면 어떤 항목이, 어떤 입력에서 틀렸는지 출력 (항목별 최대 5줄) → 버그 위치를 바로 좁힐 수 있음

입력 1개를 채점하는 방법 — ‘자리 체크표’ 4칸이 다 차면 채점

자리 체크표 (코드 이름 r_seen) 입력이 바뀐 뒤 실제로 읽은 자리에 ✓를 표시하는 4칸짜리 표. 4칸이 다 차야 ‘화면 전체를 봤다’고 인정한다



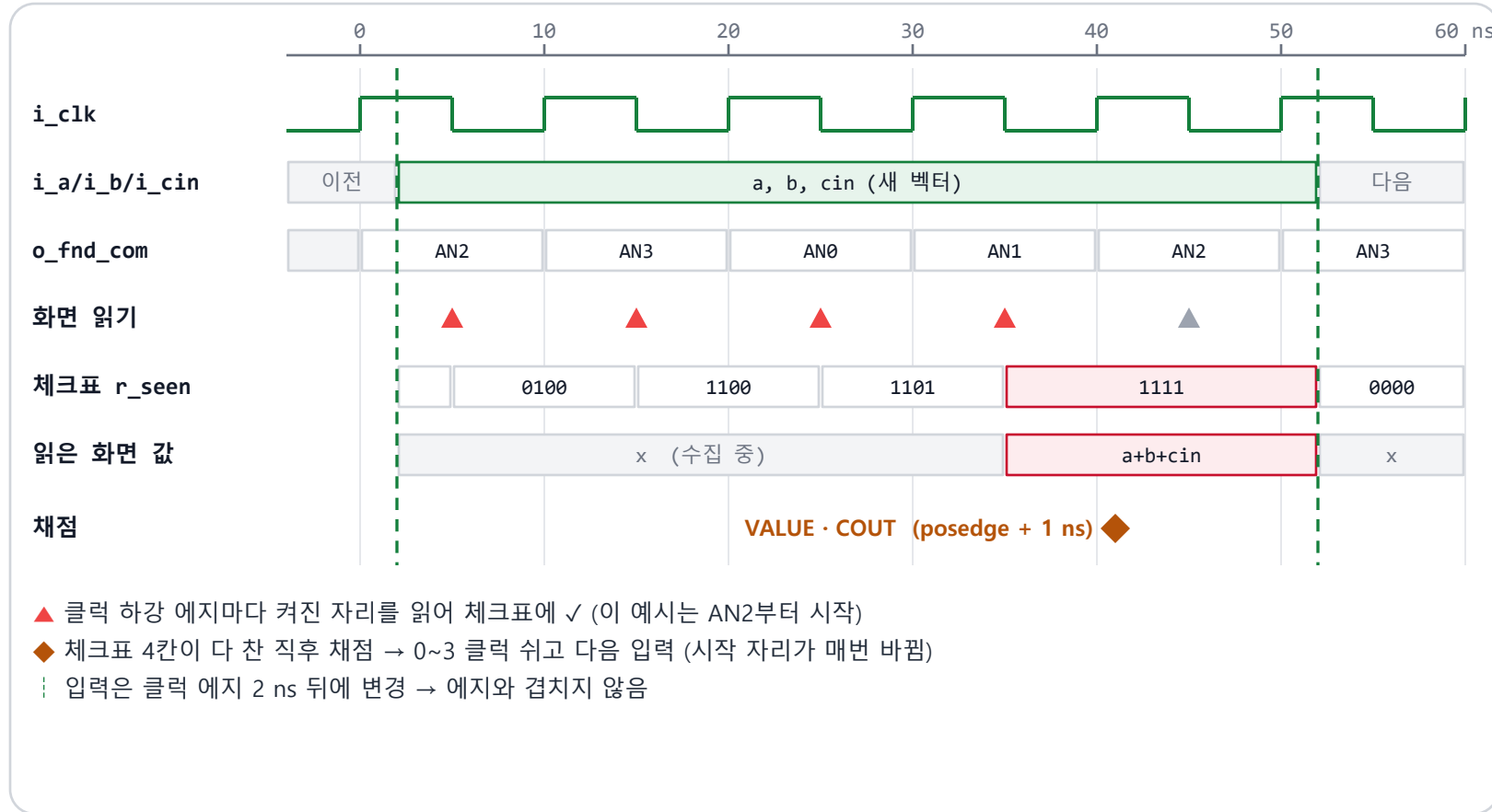
4칸이 다 차면 → 채점

화면 숫자 0510 = 정답 255 + 255 ✓ · LED = 올림 1 ✓
어느 자리부터 읽기 시작해도 4칸만 채우면 됨 → 다음 입력으로

제한 시간 안에 안 차면 → ‘자리 누락’ 오류

천의 자리가 안 커지는 버그 ②, 첫 자리에서 멈추는 버그 ③을 이렇게 잡음
순서가 틀리거나 두 자리가 같이 커지면 읽는 즉시 오류

타이밍 — 입력은 클럭 에지를 피해 바꾸고, 하강 에지에서 읽고, 4칸이 차면 채점



시작 자리가 달라도 되는 이유

판정은 “몇 ns 뒤”가 아니라 “4자리가 다 모였을 때”. 입력이 AN0가 아닌 자리에서 바뀌어도, 1 ms 스캔(V3)이어도 같은 코드로 판정한다.

그림 읽는 법

o_fnd_com 지금 켜진 자리

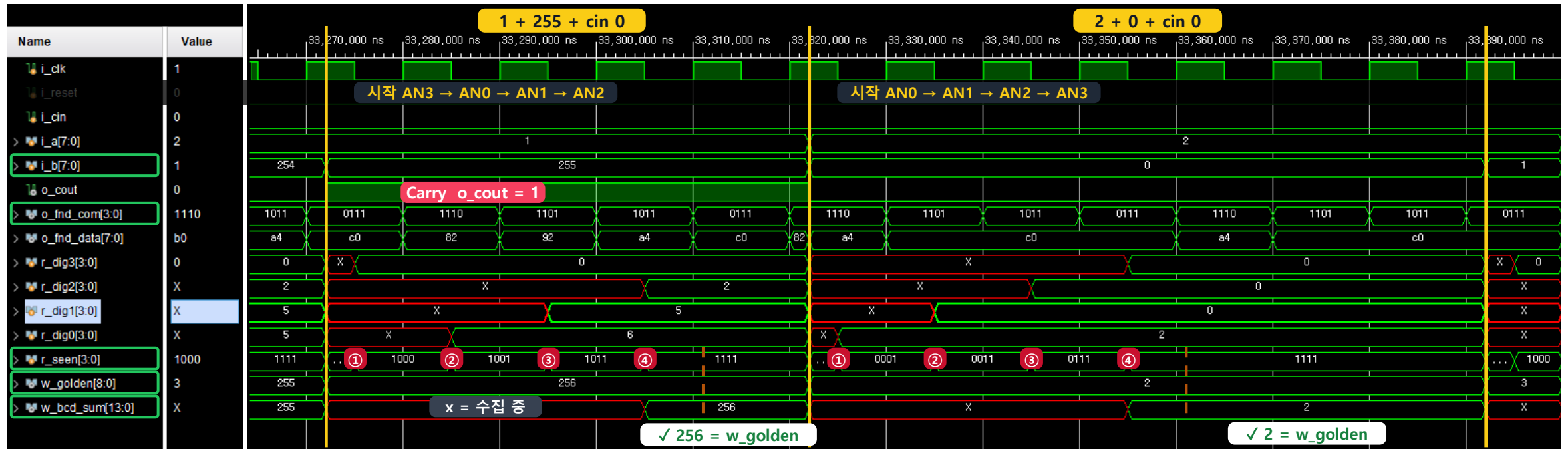
화면 읽기 ▲ 켜진 자리의 숫자를 읽는 순간

체크표 천·백·십·일 중 읽은 자리 = 1
0100 → 1100 → 1101 → 1111

읽은 화면 값 4칸이 다 차기 전엔 x

채점 ◆ 화면 값 · LED를 정답과 비교

새 TB 실행 — 131,072 전수 PASS, 4자리 수집 후 판정 동작 확인



Tcl Console — Vivado 2020.2

```
vectors = 131072
[i_cin=0] PASS (0) [i_cin=1] PASS (0)
VALUE=0 COUT=0 REFRESH=0 ONE-COLD=0 ORDER=0 FONT=0 RESET=0
[PASS] : 131072 vectors, ALL BCD CORRECT! (i_cin = 0, 1)
```

V3 Top (define V3, clock_div 1 kHz 그대로): 대표 입력 7개 PASS — xsim 재현

파형 읽는 법

입력 변경(노랑) → 자리 체크표(r_seen)를 비우고 처음부터 다시 읽음

①~④ 화면 읽기 → 켜진 자리를 읽어 체크표에 ✓. 1+255는 AN3부터, 2+0은 AN0부터 시작해도 똑같이 동작

체크표 4칸 완성 → 주황 점선에서 채점: 화면 값 = 정답 (256 = 256, 2 = 2)

0~3 클럭 쉬고 다음 입력 → 시작 자리가 매번 바뀜

결론 및 고찰

1 설계 결과

3단계 개선으로 4자리 동시 표시
1자리 표시 → 고스팅 → 클럭 분주로 해결
보드에서 최대 510까지 정상 출력



하지만 “보드에서 맞게 보인다”
≠ “검증 끝”

2 기존 TB의 문제

최종 숫자 값만 비교
화면이 어떻게 켜지는지는 보지 않음
버그 ② 천의 자리 꺼짐
버그 ④ 순서 건너뛴 → PASS로 통과
40 ns 고정 채점
스캔 속도가 바뀌면 쓸 수 없음

3 AI와 함께 한 비교 검증

- ① Claude와 기존 TB를 분석해 허점 6가지 도출
- ② 버그 5개를 심은 회로로 기존 TB와 새 TB를 같은 조건에서 비교 (Vivado)
- ③ 새 TB: 131,072개 전수 PASS, 버그 5개 모두 검출

고찰 스위치를 바꾸는 순간 값이 흔들림 → 비동기 입력이 원인, 입력 동기화(2-FF)·디바운스 방법에 대해 공부해보고 적용해보면 좋을 것 같다.

Next Step 분주 클럭 → Clock Enable 구조로 전환, 새 TB를 이후 과제의 기본 검증 틀로 사용

핵심 교훈 보드에서 맞게 보이는 것은 ‘이번 입력에서 맞았다’는 뜻일 뿐이다. 버그를 일부러 심어 TB가 잡아내는지까지 확인해야 정확한 검증을 할 수 있다는 것을 알게 되었다.